

Reviewer Pack — Communication Matrix Rank Lower Bounds ABP Size for Boolean ...

Entry 0071c0003c24 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 07:42:00 UTC

Communication Matrix Rank Lower Bounds ABP Size for Boolean Functions

Entry ID: 0071c0003c24 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-11 07:42:00 UTC

1. Conjecture

Field A (mathematical branch): Matrix Rank over Finite Fields **Field B** (complexity object): ABP Size

Statement:

For any Boolean function $f: \{0,1\}^n \times \{0,1\}^n \rightarrow \{0,1\}$, the size of the minimal ABP computing f is at least the rank of its communication matrix over $\text{GF}(2)$.

Rationale (proposer's reasoning):

The rank of the communication matrix captures linear dependencies in the function's behavior, which may impose structural constraints on the ABP's required size. ABPs inherently model computations as polynomial sequences, and low-rank matrices might correspond to more structured polynomials, potentially requiring larger ABPs to capture the necessary complexity.

Taxonomy category: BARRINGTON_ALG (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2887b1a5d77a132d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.95	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.3s

5.1 Generated Python source

```
import random
import math

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    rank = 0
    for j in range(cols):
        i_max = rank
        for i in range(rank, rows):
            if abs(matrix[i][j]) > abs(matrix[i_max][j]):
                i_max = i
        if matrix[i_max][j] == 0:
            continue
        matrix[i_max], matrix[rank] = matrix[rank], matrix[i_max]
        for i in range(rows):
            if i != rank:
                factor = -matrix[i][j] / matrix[rank][j]
                for k in range(cols):
                    matrix[i][k] += factor * matrix[rank][k]
        rank += 1
    return rank

def generate_boolean_function(n):
    return [random.randint(0, 1) for _ in range(2**n)]

def communication_matrix(f, n):
    M = [[0] * (2**n) for _ in range(2**n)]
    for i in range(2**n):
        x = i
        y = f[i]
        for j in range(n):
            if x & 1:
                y ^= f[(i >> 1) ^ (1 << j)]
            x >>= 1
        M[y][i] = 1
    return M

def abp_size(f, n):
    # Placeholder function; actual implementation depends on specific function class bounds
```

```

    return len(f)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    f = generate_boolean_function(n)
    M_f = communication_matrix(f, n)
    rank = gaussian_elimination(M_f)
    abp_size_val = abp_size(f, n)
    conjecture_holds = abp_size_val >= rank
    counterexample = "" if conjecture_holds else "mapping_undefined"
    return {
        "metric_name": "ABP Size",
        "metric_value": abp_size_val,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out without producing results, preventing evaluation of support fraction or counterexamples. | next: Run test with extended timeout and debug logging to identify crash cause

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	101757
2	preregistration	ollama_remote	qwen3:8b	0	0	24097
3	novelty	ollama_remote	qwen3:8b	0	0	20717
4	novelty	ollama_remote	qwen3:8b	0	0	14920
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12320
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8531
7	judge	ollama_remote	qwen3:8b	0	0	55445

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 237788 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in research/audit/0071c0003c24.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/0071c0003c24.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/0071c0003c24.tar.gz (if generated)